

02393 Programming in C++

Module 3:

Memory Management

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

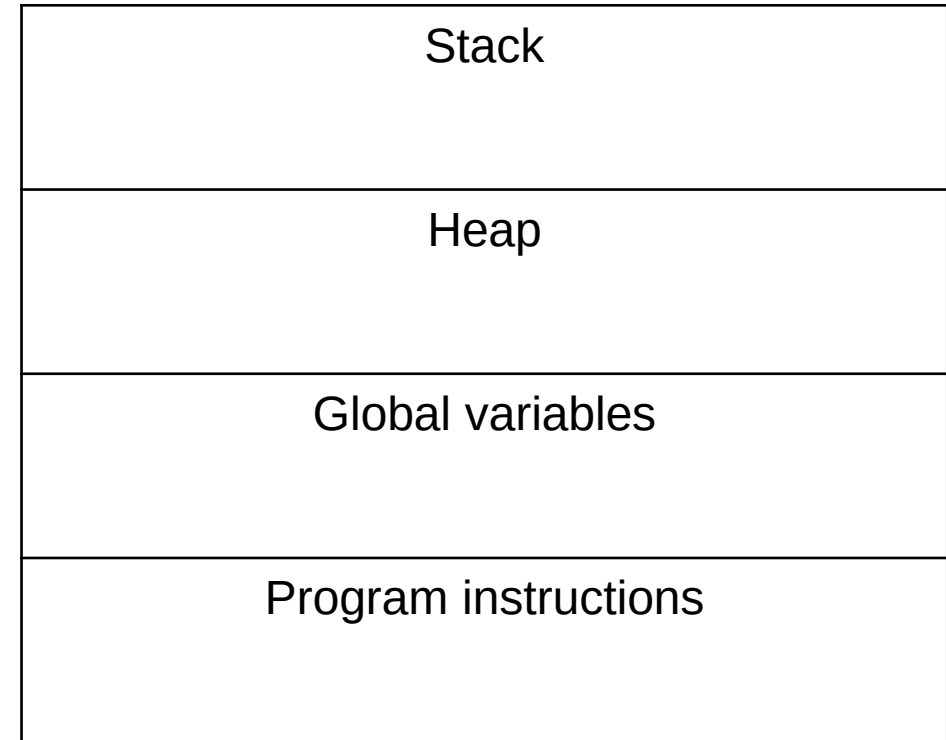
- Recap
- Stack vs heap
- Pointers vs references
- Bidimensional arrays
- Lab

A recap from the first lecture

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays, pointers, `enum`, `struct`
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - `if`, `while`, `for`, `switch`

(simplified) Memory map of a C++ program

- Stack: stores function parameters and local variables
- Heap: manual memory management with new and delete
- Global variables: accessible from anywhere in the program
- Program instructions: immutable



The function stack

```
1 int increment(int x) {  
2     int r = x + 1;  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int b = increment(a);  
9     return 0;  
10 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = ???

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted

The function stack

```
1 int increment(int x) {  
2     int r = x + 1;  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int b = increment(a);  
9     return 0;  
10 }
```

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = ???
increment()	0x1012	x = 42
	0x1016	r = ???

The function stack

```
1 int increment(int x) {  
2     int r = x + 1;  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int b = increment(a);  
9     return 0;  
10 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = ???
increment()	0x1012	x = 42
	0x1016	r = 43

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted

The function stack

```

1  int increment(int x) {
2      int r = x + 1;
3      return r;
4  }
5
6  int main() {
7      int a = 42;
8      int b = increment(a);
9      return 0;
10 }
```

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 43
increment()	0x1012	???
	0x1016	???

The function stack

```
1 int* makeArray() {  
2     int r[] = {55, 66};  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int *p = makeArray();  
9     p[0]++;  
10    return 0;  
11 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```
1 int* makeArray() {  
2     int r[] = {55, 66};  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int *p = makeArray();  
9     p[0]++;  
10    return 0;  
11 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???
makeArray()	0x1012	r[0] = 55
	0x1016	r[1] = 66
	0x1020	r = 0x1012

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```

1 int* makeArray() {
2     int r[] = {55, 66};
3     return r;
4 }
5
6 int main() {
7     int a = 42;
8     int *p = makeArray();
9     p[0]++;
10    return 0;
11 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x1012
makeArray()	0x1012	???
	0x1016	???
	0x1020	???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```

1 int* makeArray() {
2     int r[] = {55, 66};
3     return r;
4 }
5
6 int main() {
7     int a = 42;
8     int *p = makeArray();
9     p[0]++;
10    return 0;
11 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x1012
makeArray()	0x1012	???
	0x1016	???
	0x1020	???

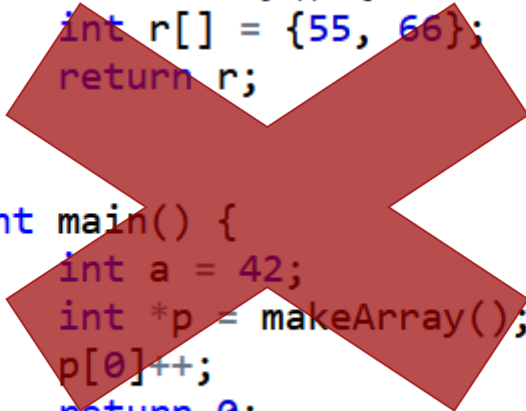
- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```

1 int* makeArray() {
2     int r[] = {55, 66};
3     return r;
4 }
5
6 int main() {
7     int a = 42;
8     int *p = makeArray();
9     p[0]++;
10    return 0;
11 }

```



Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x1012
makeArray()	0x1012	???
	0x1016	???
	0x1020	???



- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```
1 int* makeArray() {  
2     int *r = new int[2];  
3     r[0] = 55; r[1] = 66;  
4     return r;  
5 }  
6  
7 int main() {  
8     int a = 42;  
9     int *p = makeArray();  
10    p[0]++;  
11    delete[] p;  
12    return 0;  
13 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

The function stack

```

1 int* makeArray() {
2     int *r = new int[2];
3     r[0] = 55; r[1] = 66;
4     return r;
5 }
6
7 int main() {
8     int a = 42;
9     int *p = makeArray();
10    p[0]++;
11    delete[] p;
12    return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	???
0x2004	???

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???
makeArray()	0x1012	r = 0x2000

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

The function stack

```

1 int* makeArray() {
2     int *r = new int[2];
3     r[0] = 55; r[1] = 66;
4     return r;
5 }
6
7 int main() {
8     int a = 42;
9     int *p = makeArray();
10    p[0]++;
11    delete[] p;
12    return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	r[0] = 55
0x2004	r[1] = 66

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???
makeArray()	0x1012	r = 0x2000

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

The function stack

```

1 int* makeArray() {
2     int *r = new int[2];
3     r[0] = 55; r[1] = 66;
4     return r;
5 }
6
7 int main() {
8     int a = 42;
9     int *p = makeArray();
10    p[0]++;
11    delete[] p;
12    return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	r[0] = 55
0x2004	r[1] = 66

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x2000
makeArray()	0x1012	???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

The function stack

```

1  int* makeArray() {
2      int *r = new int[2];
3      r[0] = 55; r[1] = 66;
4      return r;
5  }
6
7  int main() {
8      int a = 42;
9      int *p = makeArray();
10     p[0]++;
11     delete[] p;
12     return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	r[0] = 56
0x2004	r[1] = 66

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x2000
makeArray()	0x1012	???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 1
	0x1016	y = 42
	0x1020	tmp = 1

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 42
	0x1016	y = 42
	0x1020	tmp = 1

Pointers vs references

```

1 void swap(int x, int y) {
2     int tmp = x;
3     x = y;
4     y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(a, b);
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 42
	0x1016	y = 1
	0x1020	tmp = 1

Pointers vs references

```

1 void swap(int x, int y) {
2     int tmp = x;
3     x = y;
4     y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(a, b);
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	???
	0x1016	???
	0x1020	???

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Pointers vs references

```
1 void swap(int *x, int *y) {  
2     int tmp = *x;  
3     *x = *y;  
4     *y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(&a, &b)  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b)
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b)
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 42
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b);
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 1
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b);
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 1
swap()	0x1012	???
	0x1016	???
	0x1020	???

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```
1 void swap(int &x, int &y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can also solve the problem with references
 - A reference is an alias to a variable
 - References cannot be null
 - References cannot point to a new variable
 - References must be initialized when created

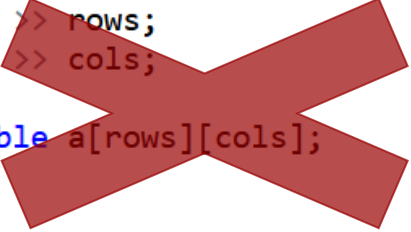
Bidimensional arrays

```
1 int a[2][2] = {{1, 2}, {3, 4}};  
2 double b[2][2];  
3  
4 cout << a[0][0] + a[1][1] << endl;  
5 b[1][0] = 3.14;
```

- We can easily define 2-dimensional arrays when their size is known at compile-time
- To access array elements, we use indexing, starting with 0
- You must pay attention not to index rows/columns beyond their last element
 - If you don't, you will end up reading or writing memory areas which belong to other variables, or to other programs!
 - Therefore, the execution may lead to unspecified results or crashes!

Bidimensional arrays

```
1 int rows, cols;
2
3 cout << "How many rows and columns?" << endl;
4 cin >> rows;
5 cin >> cols;
6
7 double a[rows][cols];
```



- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!

Bidimensional arrays

```
1 int main(){
2     int rows, cols;
3     cout << "How many rows & cols?";
4     cin >> rows; cin >> cols;
5
6     double **a = new double*[rows];
7     for (int i = 0; i < rows; i++) {
8         a[i] = new double[cols];
9     }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }
```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

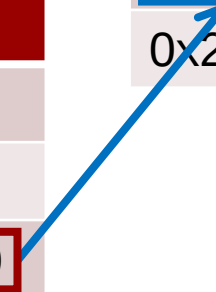
- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000

Address	Contents
0x2000	???
0x2004	???



Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0

Address	Contents
0x2000	0x3000
0x2004	???
0x3000	???
0x3004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	???
0x3004	???
0x4000	???
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0
0x1020	j = 0

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	???
0x4000	???
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0
0x1020	j = 1

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	0.0
0x4000	???
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensionanl array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1
0x1020	j = 0

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	0.0
0x4000	0.0
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1
0x1020	j = 1

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	0.0
0x4000	0.0
0x4004	0.0

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0

Address	Contents
0x2000	???
0x2004	0x4000
0x4000	0.0
0x4004	0.0

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1

Memory heap:

Address	Contents
0x2000	???
0x2004	???



Bidimensional arrays

```
1 int main(){
2     int rows, cols;
3     cout << "How many rows & cols?";
4     cin >> rows; cin >> cols;
5
6     double **a = new double*[rows];
7     for (int i = 0; i < rows; i++) {
8         a[i] = new double[cols];
9     }
10
11    for (int i = 0; i < rows; i++) {
12        for (int j = 0; j < cols; j++) {
13            a[i][j] = 0.0;
14        }
15    }
16
17    for (int i = 0; i < rows; i++) {
18        delete[] a[i];
19    }
20    delete[] a;
21    return 0;
22 }
```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = ???

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 3
 - Ask questions if something is unclear (including previous assignments)
- Next Tuesday (LAB DAY on 24 September)
 - There will be **no new topics**, just programming practice.
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 4
 - Ask questions if something is unclear (including Assignments 1 and 2)

DTU

